

Rapport technique

Scanner de vulnérabilités



Encadré par : Mr. AIRAJ Mohammed

Réalisé par :

MARCHOUAL Houssameddine

CHEIKHI Youssef

Classe : GTR 2

2024/2025

Objectif :

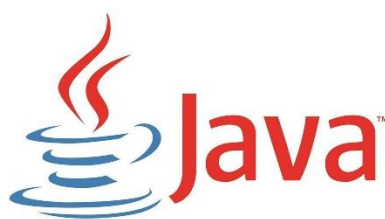
Développement d'un scanner qui identifie les vulnérabilités de sécurité dans les réseaux locaux

Fonctionnalités clés :

- **Scan de Ports** : Implémentation d'un scanner de ports pour identifier les ports ouverts sur les appareils cibles.
- **Base de Données de Vulnérabilités** : Intégration d'une base de données de vulnérabilités connues (par exemple, CVEs) pour vérifier les services découverts.
- **Rapport** : Génération des rapports sur les résultats, mettant en avant les vulnérabilités potentielles et des suggestions de remédiation.
- **Interface Utilisateur** : Création d'une interface simple en ligne de commande ou graphique pour saisir les IP cibles et voir les résultats.

Technologies :

✓ **Java**



✓ **Apache Commons Net (pour les opérations réseau)**



✓ **SQLite/MySQL (pour la base de données des vulnérabilités).**



Introduction :

Dans un monde de plus en plus interconnecté, la sécurité des réseaux locaux devient une priorité essentielle pour protéger les systèmes d'information contre les attaques malveillantes. Parmi les nombreuses menaces qui pèsent sur les infrastructures réseaux, les vulnérabilités des services accessibles via des ports ouverts constituent l'un des points d'entrée privilégiés pour les cyberattaquants. C'est dans ce contexte que le développement d'un scanner de vulnérabilités prend toute son importance.

*Le projet que nous présentons dans ce rapport vise à **développer un outil capable d'identifier les vulnérabilités de sécurité dans les réseaux locaux en scannant les ports ouverts et en analysant les services associés**. L'outil repose sur un scanner de ports qui détecte les appareils et services exposés, ainsi qu'une base de données de vulnérabilités, telle que les CVEs, permettant de vérifier la présence de failles de sécurité connues. En outre, une interface simple est mise en place pour faciliter l'interaction avec l'utilisateur et la génération de rapports détaillant les vulnérabilités détectées ainsi que des recommandations pour y remédier.*

*Ce projet utilise des technologies robustes telles que **Java** pour le développement principal, **Apache Commons Net** pour les opérations réseau et **SQLite/MySQL** pour la gestion de la base de données des vulnérabilités. L'objectif est de fournir une solution efficace et accessible pour la détection préventive des vulnérabilités, contribuant ainsi à la sécurisation des infrastructures réseaux.*

Identification des ports à scanner

Objectif :

Déterminer quels ports vont être testés pour vérifier leur ouverture.

Étapes :

1. Ports courants :

- Le scanner cible généralement les ports couramment utilisés, tels que :
 - **21** : FTP
 - **22** : SSH
 - **25** : SMTP
 - **80** : HTTP
 - **443** : HTTPS
- Une **liste de ports** (`int[] ports = {21, 22, 25, 80, 443};`) sera utilisée.

2. Saisie des ports :

l'utilisateur peut **entrer une liste personnalisée** de ports à scanner via un champ de saisie.

Établissement de la connexion avec chaque port

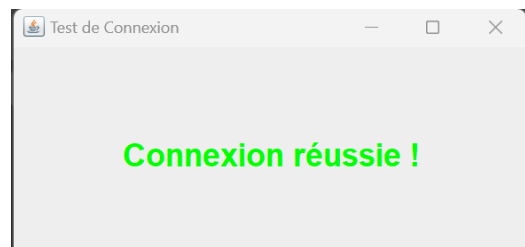
Objectif :

Tester la connectivité à un port spécifique pour vérifier s'il est **ouvert**.

Méthode :

1. Socket :

Utiliser la classe **Socket** en Java pour tenter de se connecter à une adresse IP et un port spécifique.



2. Délai de connexion :

Définir un **timeout** (ex. 2 secondes) pour ne pas bloquer le programme sur un port fermé.

```
Socket socket = new Socket();  
socket.connect(new InetSocketAddress(ip, port), timeout);
```

3. État du port :

Si la connexion réussit : le **port est ouvert**.

Si une exception est levée (timeout, refus de connexion) : le **port est fermé**.

Analyse de l'état des ports

Objectif :

Déterminer quels ports sont **ouverts** et collecter les résultats.

Étapes :

1. Boucle de scan :

Parcourir chaque port dans la liste (par exemple, via une **boucle for**).

2. Vérification de l'ouverture :

Tentative de connexion par socket.

3. Stockage des résultats :

Ajouter les résultats (ports ouverts) à une **structure de données** comme une **ArrayList** ou **HashMap**.

```

List<Integer> openPorts = new ArrayList<>();
for (int port : ports) {
    try (Socket socket = new Socket()) {
        socket.connect(new InetSocketAddress(targetIP, port), 2000);
        System.out.println("Port ouvert : " + port);
        openPorts.add(port);
    } catch (IOException e) {
        System.out.println("Port fermé : " + port);
    }
}

```

Gestion des erreurs et des exceptions

Objectif :

Garantir que le programme fonctionne correctement même en cas d'erreurs.

Méthodes de gestion :

1. Timeout :

Si le délai de connexion est dépassé, une exception `SocketTimeoutException` sera capturée.

2. Adresse invalide :

Gestion d'adresses IP incorrectes avec `UnknownHostException`.

3. Logs d'erreurs :

Afficher des messages clairs pour l'utilisateur en cas d'erreurs.

Exemple :

```

try {
} catch (SocketTimeoutException e) {
    System.out.println("Délai dépassé pour le port : " + port);
} catch (IOException e) {
    System.out.println("Erreur réseau pour le port : " + port);
}

```

Enregistrement des résultats

Objectif :

Afficher les résultats à l'utilisateur et les sauvegarder si nécessaire.

Étapes :

1. Affichage :

Résultats affichés dans une **zone de texte** en temps réel.

2. Sauvegarde facultative :

Export des résultats dans un **fichier texte** ou **PDF**.

Détection des vulnérabilités

Objectif :

Associer les ports ouverts à des vulnérabilités connues dans la base de données.

Description de la table SQLite :

- **Nom de la base** : Vulnerabilites.db
- **Colonnes** :
 - **cve_id** : Identifiant CVE (ex. CVE-2023-1234).
 - **description** : Détails de la vulnérabilité.
 - **severity** : Niveau de gravité (**High**, **Medium**).
 - **port** : Port associé à la vulnérabilité.

Exemple de requête SQL :

Pour rechercher les vulnérabilités associées au port **80** :

```
SELECT * FROM vulnerabilites WHERE port = 80;
```

Exemple en Java (avec JDBC) :

```
Connection conn =  
DriverManager.getConnection("jdbc:sqlite:Vulnerabilites.db");  
String query = "SELECT * FROM vulnerabilites WHERE port = ?";  
PreparedStatement stmt = conn.prepareStatement(query);  
stmt.setInt(1, port);  
ResultSet rs = stmt.executeQuery();  
  
while (rs.next()) {  
    System.out.println("CVE : " + rs.getString("cve_id"));  
    System.out.println("Description : " + rs.getString("description"));  
    System.out.println("Gravité : " + rs.getString("severity"));  
}
```

	id	port	description	severity
1	1	80	HTTP - Vulnérabilité au Cross-Site Scripting (XSS)	High
2	2	443	HTTPS - Vulnérabilité à la configuration SSL/TLS	Medium
3	3	3306	MySQL - Vulnérabilité aux attaques SQL Injection	High
4	4	21	FTP - Vulnérabilité au déni de service (DoS)	High
5	5	23	Telnet - Vulnérabilité de sécurité critique	High
6	6	25	SMTP - Vulnérabilité au spam (Mail Relay)	Medium
7	7	53	DNS - Vulnérabilité à l'empoisonnement de cache	High
8	8	110	POP3 - Vulnérabilité à l'injection de commandes	Medium
9	9	143	IMAP - Vulnérabilité de fuite d'informations	Medium
10	10	3389	RDP - Remote Desktop Protocol	High
11	11	5900	VNC - Vulnérabilité à l'accès non autorisé	High
12	12	6379	Redis - Vulnérabilité d'exécution de commandes à distance	High
13	13	8080	HTTP Proxy - Vulnérabilité au script malveillant	Medium
14	14	27017	MongoDB - Vulnérabilité à l'injection de commandes	High
15	15	5432	PostgreSQL - Vulnérabilité d'accès non autorisé	Medium

Génération de rapports

Objectif :

Créer un **résumé clair et structuré** des ports scannés et des vulnérabilités détectées.

Structure du rapport :

- **Ports ouverts** : Liste des ports détectés comme ouverts.
- **Vulnérabilités associées** : Liste des vulnérabilités pour chaque port.

Exemple de rapport :

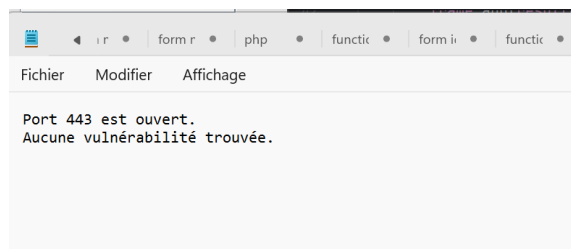
Scanner de Vulnérabilités - Résultats

Port ouvert : 80 (HTTP)

- CVE-2023-1234 : Injection XSS (Gravité : High)

Port ouvert : 443 (HTTPS)

- CVE-2023-5678 : Protocole TLS vulnérable (Gravité : Medium)



Présentation de l'interface graphique

Composants de l'interface :

1. Champ de saisie :

Pour entrer l'**adresse IP** cible et éventuellement une **liste de ports**.

2. Bouton "Scanner" :

Lance le processus de scan.

3. **Zone de texte :**

Affiche en **temps réel** les résultats :

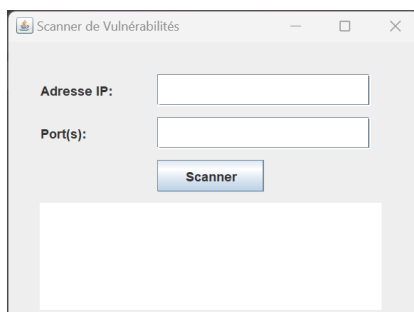
- Ports ouverts.
- Vulnérabilités associées.

Exemple de code (interface simple avec Swing) :

```
JFrame frame = new JFrame("Scanner de Vulnérabilités");
JTextField ipField = new JTextField("Entrez l'adresse IP");
JButton scanButton = new JButton("Scanner");
JTextArea resultArea = new JTextArea();

scanButton.addActionListener(e -> {
    resultArea.append("Scan en cours...\n");
});

frame.add(ipField, BorderLayout.NORTH);
frame.add(scanButton, BorderLayout.CENTER);
frame.add(new JScrollPane(resultArea), BorderLayout.SOUTH);
frame.setSize(400, 300);
frame.setVisible(true);
```



Conclusion

Dans le cadre de ce projet "**Scanner de Vulnérabilités**", nous avons développé un outil capable de **scanner les ports ouverts** sur un réseau local et d'identifier les vulnérabilités connues grâce à une base de données dédiée.

Dans un premier temps, nous avons utilisé une **connexion par sockets** pour vérifier l'état des ports courants, ce qui nous a permis d'identifier rapidement et efficacement les ports ouverts. Ensuite, en effectuant des **requêtes SQL** sur une base de données contenant des informations sur les vulnérabilités (CVE), nous avons pu détecter les failles de sécurité associées aux services présents sur ces ports.

Nous avons également mis en place une **interface utilisateur simple et intuitive**, comprenant un champ de saisie pour entrer l'adresse IP cible, un bouton pour lancer le scan et une zone d'affichage pour visualiser les résultats en temps réel. Cette approche rend l'outil accessible aussi bien aux administrateurs réseau qu'aux étudiants souhaitant analyser la sécurité d'un réseau local.

Enfin, nous avons intégré une **génération de rapports structurés** permettant de résumer les ports scannés et les vulnérabilités détectées. Cette fonctionnalité facilite l'évaluation des risques et constitue une aide précieuse pour renforcer la sécurité des systèmes.

Grâce à ce projet, nous avons contribué à l'amélioration des outils d'analyse des vulnérabilités, en proposant une solution efficace pour la **surveillance et la protection des réseaux locaux**. Ce travail constitue une **base solide** pour des évolutions futures, comme l'ajout de nouvelles fonctionnalités ou l'intégration d'une mise à jour automatique des données CVE.